# Terraform Part 6

## State & Backends

Professional Terraform Notes | Beginner to Advanced | AWS S3 | State Locking | Production Troubleshooting | Interview Preparation

Language format: English technical notes with explanations in simple, everyday Kannada. Each topic follows your five requested headings.

ಈ Part ನಲ್ಲಿ Terraform State, Remote Backend, S3, State Locking, Refresh, Import ಮತ್ತು State Recovery ಬಗ್ಗೆ ಕಲಿಯೋಣ. ನಿಜವಾದ AWS ಮತ್ತು Jenkins CI/CD projectಗಳಲ್ಲಿ state ಸಮಸ್ಯೆ ಬಂದಾಗ ಹೇಗೆ troubleshoot ಮಾಡಬೇಕು ಅನ್ನೋದಕ್ಕೂ ಇಲ್ಲಿ ಉದಾಹರಣೆಗಳಿವೆ.

### Topics covered


1. Terraform State — terraform.tfstate
2. Why Terraform State Is Required
3. Local State vs Remote State
4. Terraform Backends
5. AWS S3 Remote Backend
6. State Locking
7. State Refresh and Drift Detection
8. Terraform Import
9. State File Backup and Recovery
10. State Commands and State Manipulation
11. Workspace and State Separation
12. Real-Time Production Scenarios and Interview Questions


## Topic 1: Terraform State

### 1. Definition — What is Terraform State?

Terraform State is a data structure that records the relationship between Terraform configuration addresses and the real infrastructure objects managed by Terraform.

By default, Terraform stores this information in a file named terraform.tfstate.

State may contain:

- Resource IDs, such as EC2 instance IDs.
- Resource attributes, such as instance types and IP addresses.
- Resource addresses and their association with real infrastructure.
- Dependency-related information and other provider-managed values.
- Sensitive values, depending on the resources and configuration.

Example: Terraform may record that aws_instance.web corresponds to the actual AWS instance i-0123456789abcdef0.

## 2. Purpose — Why do we use Terraform State?

Terraform State is important because it helps Terraform:

- Identify resources it already manages.
- Compare desired configuration with recorded infrastructure information.
- Calculate the changes required during planning.
- Track resources across subsequent operations.
- Manage dependencies and resource addresses.
- Store information needed for updates, replacement, and deletion.

## 3. Working — How does Terraform State work?

1. You write an EC2 resource in main.tf.
2. You execute terraform apply.
3. Terraform uses the AWS provider to create the EC2 instance.
4. Terraform records the association between the resource address and the AWS instance in state.
5. During later operations, Terraform uses configuration, state, and information retrieved from AWS to determine what changes are needed.

State is not simply a copy of the Terraform code or a guaranteed real-time representation of every cloud attribute. Terraform must refresh relevant information from the provider during appropriate operations.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** State ಅಂದರೆ Terraform ತಾನು ಯಾವ resources create ಮಾಡಿದೆ, ಆ resources AWSನಲ್ಲಿ ಯಾವ IDs ಹೊಂದಿವೆ ಅನ್ನೋದನ್ನು ದಾಖಲಿಸುವ ಮಾಹಿತಿ. ಮುಂದಿನ ಬಾರಿ terraform plan ಮಾಡಿದಾಗ ಈಗಾಗಲೇ ಏನಿದೆ, ಏನು ಬದಲಾವಣೆ ಬೇಕು ಅಂತ ತಿಳಿದುಕೊಳ್ಳಲು ಇದು ಸಹಾಯ ಮಾಡುತ್ತದೆ.

## 4. Practical Example — View Terraform State

Suppose your Terraform configuration creates an EC2 instance.

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  tags = {
    Name = "web-server"
  }
}
```

After applying the configuration, run:

```
terraform state list
```

Example output:

```
aws_instance.web
```

To inspect the resource recorded in state:

```
terraform state show aws_instance.web
```

The output may include the instance ID, instance type, AMI, and other recorded attributes.

Important: State output can contain sensitive information. Avoid publishing it in GitHub, Jenkins logs, screenshots, or unsecured storage.

### 5. Scenario-Based Interview Questions

#### Q1. What is Terraform State, and why is it required?

**Answer:** Terraform State records the association between Terraform resource addresses and managed infrastructure objects. Terraform uses it, together with configuration and provider information, to plan and manage infrastructure changes.

#### Q2. What happens if the state file is deleted?

**Answer:** Terraform may lose its record of managed resources. The actual AWS resources may still exist, but Terraform may no longer associate them with the configuration. I would stop further changes, check remote state and backups, and recover the correct state before running another apply.

#### Q3. Is Terraform State the same as the actual AWS infrastructure?

**Answer:** No. State records Terraform's knowledge of managed resources. Actual infrastructure exists in AWS, and the two can differ when resources change outside Terraform or state becomes outdated.

## Topic 2: Why Terraform State Is Required

### 1. Definition — What is the role of State?

Terraform State provides the resource identity and recorded information needed for Terraform to manage infrastructure across multiple operations.

### 2. Purpose — Why is it important?

Without a correct state association, Terraform may not know that an existing AWS resource is already managed by a particular resource address.

State supports:

- Resource tracking.
- Change planning.
- Dependency management.
- Mapping configuration to real resources.
- Controlled updates and deletion.
- Team collaboration when stored in a properly configured remote backend.

### 3. Working — How does it work?

Suppose the configuration declares one EC2 instance.

On the first apply, Terraform creates the instance and records its identity in state.

On a later plan, Terraform checks the configuration against the recorded state and refreshes relevant infrastructure information. If the desired configuration still matches, the plan may show no changes.

If the configuration changes the instance type, Terraform determines whether an in-place update or replacement is needed.

### 4. Practical Example — No Changes

```
terraform plan
```

Example output:

```
No changes. Your infrastructure matches the configuration.
```

This means Terraform has determined that no changes are required by the current plan. It does not prove that every aspect of the environment is secure or that no unrelated resources exist.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** State ಸರಿಯಾಗಿದ್ದರೆ Terraform ಈಗಾಗಲೇ create ಮಾಡಿದ resources ಗುರುತಿಸಬಹುದು. ಆಗ ಅನಗತ್ಯವಾಗಿ ಹೊಸ EC2 instance create ಮಾಡುವ ಸಾಧ್ಯತೆ ಕಡಿಮೆಯಾಗುತ್ತದೆ.

### 5. Scenario-Based Interview Questions

**Q1. Why does Terraform need State when the configuration is already available?**

**Answer:** Configuration describes the desired infrastructure, but state records which real objects are associated with Terraform resource addresses. Terraform uses both to calculate appropriate changes.

**Q2. Terraform creates duplicate infrastructure after a state problem. Why might this happen?**

**Answer:** Terraform may not have the correct association with the existing infrastructure. I would inspect the state, verify the resource addresses, and recover or import resources as appropriate before applying changes.

**Q3. Can you safely edit the state file manually?**

**Answer:** Direct manual editing is risky because state has a specific structure and contains resource associations. I would prefer supported commands such as terraform state mv, terraform state rm, or an appropriate import workflow.

## Topic 3: Local State vs Remote State

### 1. Definition — What is Local and Remote State?

Local State: Terraform stores state in a file on the machine where Terraform runs, usually terraform.tfstate.

Remote State: Terraform stores state in a remote backend, such as an AWS S3 bucket, so authorized users and automation systems can access the shared state.

### 2. Purpose — Why do we use them?

Local state is convenient for learning, testing, and small personal projects.

Remote state is generally preferable for team projects and CI/CD because it provides centralized storage and can support locking, access control, backup, and recovery mechanisms.

### 3. Working — How do they work?

With local state, Terraform reads and updates the local state file.

With remote state, Terraform communicates with the configured backend to read and write state. Backend capabilities depend on the backend type and its configuration.

| Feature | Local State | Remote State |
|---|---|---|
| Storage | Local working directory | Remote storage system |
| Team collaboration | Difficult to coordinate | Centralized state access |
| CI/CD integration | Requires careful state handling | Well suited to shared pipelines |
| Access control | Depends on machine and file permissions | Can use backend-specific IAM and policies |
| Locking | Limited to applicable local mechanisms | Depends on backend locking support |
| Recovery | Requires local backups | Depends on backend versioning and backup setup |

### 4. Practical Example — Local to Remote State

Initially, a project might use local state:

```
terraform-project/
|-- main.tf
|-- provider.tf
`-- terraform.tfstate
```

For a team project, you can configure an S3 backend so Terraform stores state centrally.

After configuring the backend, initialize or migrate using the appropriate Terraform initialization workflow. Always back up existing state before migration.

### 5. Scenario-Based Interview Questions

#### Q1. Why would you use remote state in a Jenkins CI/CD project?

**Answer:** Jenkins needs to work with the same infrastructure state as authorized engineers. A remote backend centralizes state storage and can provide locking and access control, helping avoid conflicting operations.

#### Q2. Is remote state automatically secure?

**Answer:** No. I would configure appropriate IAM permissions, encryption, restricted bucket access, secure network practices where applicable, and recovery protections.

#### Q3. Can multiple engineers share the same local state file safely?

**Answer:** Sharing a local file manually is risky. For team infrastructure, I would use a suitable remote backend and its supported coordination mechanisms.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** Local state ಅಂದರೆ state file ನಮ್ಮ computerನಲ್ಲಿ ಇರುತ್ತದೆ. Remote state ಅಂದರೆ AWS S3 ತರಹದ central storageನಲ್ಲಿ ಇರುತ್ತದೆ. Jenkins ಮತ್ತು team members ಒಂದೇ infrastructure ಮೇಲೆ ಕೆಲಸ ಮಾಡುವಾಗ remote state ಹೆಚ್ಚು ಅನುಕೂಲಕರ.

## Topic 4: Terraform Backends

### 1. Definition — What is a Backend?

A Terraform backend determines where Terraform stores its state and, depending on the backend, how it handles state operations and locking.

Common backend options include:

- Local backend.
- Amazon S3 backend.

- HCP Terraform and Terraform Enterprise remote execution/state capabilities.
- Other supported remote backends.

## 2. Purpose — Why do we use Backends?

Backends help determine how state is stored, shared, secured, and accessed.

For team projects, an appropriate remote backend can centralize state and support operational controls such as locking and recovery.

## 3. Working — How does a Backend work?

Terraform initializes the configured backend during terraform init.

The backend then handles the storage operations for state. For remote backends, Terraform can read and update state remotely rather than relying on a state file stored only in the working directory.

Changing the backend configuration may require reinitialization or state migration.

## 4. Practical Example — Configure an S3 Backend

```
terraform {
  backend "s3" {
    bucket       = "my-terraform-state-bucket"
    key          = "dev/application/terraform.tfstate"
    region       = "us-east-1"
    use_lockfile = true
  }
}
```

This example uses the S3 backend's native lock-file mechanism. Replace the bucket name with your actual bucket.

Initialize:

```
terraform init
```

For an existing project migrating from local state, Terraform may ask whether to migrate the existing state. Back up the current state first and review the migration prompt carefully.

Note: The S3 bucket must already exist. Backend initialization occurs before Terraform can use the configuration to manage ordinary resources.

## 5. Scenario-Based Interview Questions

### Q1. What is the difference between a provider and a backend?

**Answer:** A provider manages resources through a platform's API. A backend determines where Terraform state is stored and how Terraform accesses it.

### Q2. What happens if the configured S3 backend bucket does not exist?

**Answer:** Backend initialization or state access fails. I would verify the bucket name, AWS account, region, permissions, and bucket existence.

### Q3. Why can changing a backend configuration require terraform init again?

**Answer:** Terraform must initialize the new backend configuration and may need to migrate existing state to the new location.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** Provider AWS resources create ಮಾಡಲು ಮತ್ತು manage ಮಾಡಲು ಸಹಾಯ ಮಾಡುತ್ತದೆ. Backend state file ಎಲ್ಲಿ save ಆಗಬೇಕು ಅಂತ ನಿರ್ಧರಿಸುತ್ತದೆ. S3 backend ಬಳಸಿದರೆ state ಅನ್ನು S3 bucketನಲ್ಲಿ ಇಡಬಹುದು.

# Topic 5: AWS S3 Remote Backend

## 1. Definition — What is an S3 Backend?

The S3 backend stores Terraform state in an Amazon S3 object. The object key identifies the state location inside the bucket.

For example:

```
Bucket: company-terraform-state
Key:    dev/eks/terraform.tfstate
Region: us-east-1
```

## 2. Purpose — Why do we use an S3 Backend?

- Store state centrally.
- Share infrastructure state across authorized users and pipelines.
- Use AWS IAM to control access.
- Enable encryption and recovery features.
- Integrate Terraform with Jenkins and other automation systems.
- Support state locking through the backend's supported mechanism.

## 3. Working — How does S3 State work?

1. Create an S3 bucket with suitable security settings.
2. Configure the S3 backend in Terraform.
3. Give the Terraform execution identity the required S3 permissions.
4. Initialize the backend.
5. Terraform reads and writes the state object during supported operations.
6. Enable a suitable locking mechanism and recovery controls.

A production bucket should normally have public access blocked, encryption enabled, tightly scoped permissions, and versioning configured to help recover earlier object versions.

## 4. Practical Example — Complete Backend Configuration

```
terraform {
  backend "s3" {
    bucket       = "company-terraform-state"
    key          = "dev/eks/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
}
```

Initialize the backend:

```
terraform init
terraform plan
```

The S3 bucket must exist before backend initialization. The identity used by Terraform must have the necessary permissions for the state object and the lock file.

Important S3 backend considerations

- Bucket: Central state storage location.
- Key: Object path for the particular state.
- Region: Region where the bucket exists.
- Encryption: Protects stored state.
- Versioning: Helps recover previous versions of the state object.
- Lock file: Coordinates supported concurrent operations when use_lockfile = true.

For a production project, state access should be limited to authorized identities. Terraform state can contain sensitive data even when outputs are marked sensitive.

### 5. Scenario-Based Interview Questions

**Q1. Your Jenkins pipeline cannot access the S3 backend. What would you check?**

**Answer:** I would check the bucket and key, bucket region, AWS identity, S3 permissions, backend configuration, encryption requirements, and whether the pipeline is using the intended AWS account.

**Q2. Your state object was accidentally overwritten. How could you recover it?**

**Answer:** If bucket versioning was enabled before the incident, I would identify the correct previous version, preserve the current state for investigation, and restore state carefully after confirming that the version matches the actual infrastructure.

**Q3. Can the same S3 bucket store multiple Terraform states?**

**Answer:** Yes. Different state keys can store different configurations, such as development and production. Each configuration must use the correct key and appropriate access controls.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** S3 bucket ಅನ್ನು Terraform state ಇಡಲು ಬಳಸಬಹುದು. Bucket ಒಳಗೆ key ಮೂಲಕ ಪ್ರತಿ projectನ stateಗೆ ಬೇರೆ path ಕೊಡಬಹುದು. Jenkins ಮತ್ತು developersಗೆ ಅಗತ್ಯವಿರುವಷ್ಟು permission ಮಾತ್ರ ಕೊಡಬೇಕು.

## Topic 6: State Locking

### 1. Definition — What is State Locking?

State locking is a mechanism that prevents conflicting Terraform operations from modifying the same state simultaneously when the configured backend supports locking.

### 2. Purpose — Why do we use State Locking?

Imagine two engineers run terraform apply against the same state at nearly the same time. Their changes could conflict or produce unsafe results.

Locking helps coordinate operations that need exclusive access to state.

### 3. Working — How does State Locking work?

1. Terraform begins an operation that requires a state lock.
2. The backend attempts to acquire the lock.
3. If the lock is acquired, the operation proceeds.
4. If another operation already holds the lock, Terraform may wait or return a locking error.
5. After the operation finishes, Terraform releases the lock.

The exact behavior depends on the backend and locking mechanism.

S3 locking options

For the S3 backend, current Terraform supports native lock files using:

```
use_lockfile = true
```

The older DynamoDB-based locking mechanism is deprecated in current Terraform documentation. Existing projects may still use DynamoDB locking, but new configurations should generally consider native S3 lock files.

## 4. Practical Example — Enable S3 Locking

```
terraform {
  backend "s3" {
    bucket       = "company-terraform-state"
    key          = "prod/eks/terraform.tfstate"
    region       = "us-east-1"
    use_lockfile = true
  }
}
```

Then run:

```
terraform init
terraform plan
terraform apply
```

Ensure the executing identity has the required permissions for both the state object and the lock file.

If Terraform reports a lock error

First, verify whether another Terraform operation is running.

```
terraform plan
```

If the state is genuinely locked by a process that has terminated, investigate the lock and state before taking action. Do not remove a lock simply because it is inconvenient.

If necessary, Terraform supports:

```
terraform force-unlock LOCK_ID
```

Use this only after verifying that the lock is stale and no active operation owns it. The lock ID must be the actual ID supplied by Terraform.

## 5. Scenario-Based Interview Questions

### Q1. Jenkins reports that the Terraform state is locked. What would you do?

**Answer:** I would check whether another Jenkins build or engineer is running Terraform against the same state. I would wait for a legitimate operation to finish. If the lock appears stale, I would verify the owner and operation status before considering force-unlock.

### Q2. Does state locking prevent all infrastructure changes outside Terraform?

**Answer:** No. It coordinates Terraform operations that use the relevant locking mechanism. It does not prevent an administrator from changing AWS resources directly through the Console or API.

### Q3. What happens if you force-unlock while another Terraform operation is running?

**Answer:** It can allow conflicting operations and risk state inconsistency. I would never force-unlock without confirming that the existing operation is no longer active.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** State lock ಅಂದರೆ ಒಂದೇ state ಮೇಲೆ ಒಂದೇ ಸಮಯದಲ್ಲಿ ಎರಡು Terraform operations conflict ಆಗದಂತೆ ತಡೆಯುವ ವ್ಯವಸ್ಥೆ. Jenkins build ಇನ್ನೂ ನಡೆಯುತ್ತಿರುವಾಗ ಮತ್ತೊಂದು build ಅದೇ state ಮೇಲೆ apply ಮಾಡಿದರೆ ಸಮಸ್ಯೆ ಬರಬಹುದು. Lock ಅನ್ನು ಪರಿಶೀಲಿಸದೆ delete ಮಾಡಬಾರದು.

# Topic 7: State Refresh and Drift Detection

### 1. Definition — What is State Refresh?

State refresh is the process of updating Terraform's recorded information about managed infrastructure using information retrieved from the provider.

Drift detection identifies differences between the desired configuration and actual infrastructure.

### 2. Purpose — Why do we use Refresh?

Refresh helps Terraform account for changes made outside Terraform, such as:

- An administrator changing an EC2 instance type.
- A security group rule being changed in AWS.
- An instance being terminated manually.
- An autoscaler changing a managed attribute.

### 3. Working — How does it work?

During planning, Terraform typically refreshes information about managed resources and compares the result with the configuration.

A typical command is:

```
terraform plan
```

Terraform may detect a difference and propose a change to reconcile infrastructure with the configuration.

The older standalone terraform refresh command is deprecated. For modern workflows, use terraform plan or, when there is a specific need to update state without proposing configuration changes, terraform apply -refresh-only.

### 4. Practical Example — Detect Manual Changes

Terraform configuration:

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  tags = {
    Name = "application-server"
  }
}
```

An administrator manually changes the instance type in AWS to t3.small.

Run:

```
terraform plan
```

Terraform may show a proposed change to return the instance to the configured type.

If the manual change is intentional, update the Terraform configuration to reflect the approved new setting. If it is unauthorized, follow your operational process to restore the intended configuration.

Refresh-only workflow

```
terraform plan -refresh-only
terraform apply -refresh-only
```

The first command previews state updates based on refreshed information. The second applies those state updates without making the usual configuration-driven infrastructure changes.

Review the output carefully before applying.

### 5. Scenario-Based Interview Questions

**Q1. Your AWS Console shows a different instance type from Terraform configuration. How would you troubleshoot it?**

**Answer:** I would run a plan, inspect the detected drift, identify why the instance type changed, and decide whether the change should be adopted into code or reverted through an approved apply.

**Q2. Does terraform plan automatically fix drift?**

**Answer:** No. Planning previews proposed changes. A normal apply executes the planned configuration changes after approval.

**Q3. What is the difference between a normal plan and a refresh-only plan?**

**Answer:** A normal plan evaluates changes needed to align infrastructure with configuration. A refresh-only plan focuses on updating Terraform's recorded state based on what the provider reports.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** AWS Consoleನಲ್ಲಿ ಯಾರಾದರೂ resource change ಮಾಡಿದರೆ Terraform code ಮತ್ತು actual infrastructure ನಡುವೆ ವ್ಯತ್ಯಾಸ ಬರಬಹುದು. ಅದನ್ನೇ drift ಅಂತ ಕರೆಯುತ್ತೇವೆ. terraform plan ಮಾಡಿದಾಗ ಆ ವ್ಯತ್ಯಾಸ ಗೊತ್ತಾಗಬಹುದು. Plan ಪರಿಶೀಲಿಸದೆ apply ಮಾಡಬಾರದು.

## Topic 8: Terraform Import

### 1. Definition — What is Terraform Import?

Terraform import is the process of associating an existing infrastructure object with a Terraform resource address in state.

It is useful when resources were created manually or need to be brought under Terraform management.

### 2. Purpose — Why do we use Import?

- Adopt manually created AWS resources into Terraform.
- Avoid unnecessarily recreating existing infrastructure.
- Migrate from manual provisioning to Infrastructure as Code.
- Bring resources under a consistent change-management process.
- Integrate existing infrastructure into an established Terraform project.

### 3. Working — How does Import work?

1. Identify the existing resource and its unique ID.
2. Write the corresponding Terraform resource configuration.
3. Import the resource into the intended Terraform resource address.

4. Run terraform plan.
5. Reconcile the configuration with the actual infrastructure.
6. Apply only after reviewing the plan.

Import associates the existing object with Terraform state. It does not automatically create a complete Terraform configuration in the traditional CLI workflow.

### 4. Practical Example — Import an Existing EC2 Instance

Suppose an EC2 instance was created manually in AWS.

Create a resource block:

```
resource "aws_instance" "existing" {
  # Add configuration matching the existing instance.
  # Confirm required arguments using the AWS provider documentation.
}
```

Import the existing instance:

```
terraform import aws_instance.existing i-0123456789abcdef0
```

The instance ID is illustrative. Replace it with the real instance ID.

Then inspect the resource:

```
terraform state show aws_instance.existing
terraform plan
```

Update the configuration to reflect the intended settings and resolve any differences shown by the plan before applying.

Important: Importing does not mean Terraform will never modify or replace that resource. Future plans may propose changes based on the configuration.

### 5. Scenario-Based Interview Questions

**Q1. Your company has an EC2 instance created manually, and management wants it managed through Terraform. What would you do?**

**Answer:** I would create the matching resource configuration, import the instance into the appropriate resource address, inspect the state, and reconcile the configuration with the existing infrastructure through reviewed plans.

**Q2. Does terraform import create a new EC2 instance?**

**Answer:** No. It associates an existing supported object with Terraform state.

**Q3. After importing, Terraform proposes replacing the instance. Why?**

**Answer:** The configuration may not match the existing instance, or a required attribute may force replacement. I would investigate the plan and correct the configuration before considering any replacement.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** AWSನಲ್ಲಿ ಈಗಾಗಲೇ instance ಇದೆ ಅಂದರೆ ಅದನ್ನೇ Terraform ಮೂಲಕ manage ಮಾಡಲು import ಬಳಸಬಹುದು. Import ಮಾಡಿದ ತಕ್ಷಣ instance create ಆಗುವುದಿಲ್ಲ. Existing resource ಮತ್ತು Terraform state ನಡುವೆ link ಆಗುತ್ತದೆ. ನಂತರ configuration ಸರಿಯಾಗಿದೆಯಾ ಅಂತ terraform plan ಮೂಲಕ ಪರಿಶೀಲಿಸಬೇಕು.

## Topic 9: Terraform State Backup and Recovery

## 1. Definition — What is State Recovery?

State recovery is the process of restoring or reconstructing the correct Terraform state after accidental deletion, corruption, an incorrect state operation, or another state-management failure.

## 2. Purpose — Why do we need State Recovery?

Terraform state is critical for managing existing infrastructure. Losing it can cause Terraform to lose its record of resource associations.

Recovery helps restore those associations and reduce the risk of duplicate infrastructure, accidental deletion, or inconsistent state.

## 3. Working — How does State Recovery work?

The recovery procedure depends on where state is stored.

For local state:

1. Stop further Terraform operations.
2. Preserve the damaged or current files for investigation.
3. Locate a valid backup.
4. Restore the correct state.
5. Inspect resources and run a plan.

For an S3 backend:

1. Stop concurrent Terraform operations.
2. Identify the affected bucket, object key, and incident time.
3. Preserve the current state version.
4. Locate a suitable prior S3 object version or backup.
5. Restore state through a controlled process.
6. Verify the resource associations and review the plan.

State recovery must be coordinated because restoring an old state version can discard knowledge of infrastructure changes made afterward.

## 4. Practical Example — Backup Local State

Before a migration or sensitive state operation:

```
terraform state pull > terraform-state-backup.json
```

This command retrieves the current state snapshot from the configured backend and writes it to a local file. Protect the resulting file as sensitive data.

For an S3 backend, versioning should be enabled in advance so previous object versions can be recovered when necessary.

A recovery procedure should verify that the restored state matches the actual infrastructure before normal operations resume.

## 5. Scenario-Based Interview Questions

**Q1. Your terraform.tfstate file was accidentally deleted. What would you do first?**

**Answer:** I would stop further applies, check for a remote backend and valid backups, preserve any remaining files, and restore the correct state. If no valid state exists, I would carefully reconstruct resource associations rather than blindly applying the configuration.

**Q2. Can you restore an old S3 state version without checking current infrastructure?**

**Answer:** No. The restored state may not include more recent resource changes. I would preserve the current version, compare timestamps and resource associations, and verify the actual infrastructure before resuming operations.

**Q3. Why is S3 versioning useful for Terraform state?**

**Answer:** It preserves previous versions of the state object, which can help recover from accidental overwrites or corruption. Versioning is a recovery aid, not a substitute for correct permissions and a tested recovery procedure.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** State file delete ಅಥವಾ corrupt ಆದರೆ AWS resources ಕೂಡ delete ಆಗಿವೆ ಅಂತ ಅರ್ಥವಲ್ಲ. Stateನಲ್ಲಿರುವ record ಮಾತ್ರ ಕಳೆದುಹೋಗಿರಬಹುದು. ಮೊದಲು operations ನಿಲ್ಲಿಸಿ, backup ಅಥವಾ S3 previous version ಪರಿಶೀಲಿಸಿ, ಸರಿಯಾದ state restore ಮಾಡಬೇಕು. Backup ಇಲ್ಲದಿದ್ದರೆ import ಅಥವಾ ಬೇರೆ recovery ವಿಧಾನ ಬೇಕಾಗಬಹುದು.

# Topic 10: Terraform State Commands and State Manipulation

## 1. Definition — What are Terraform State Commands?

Terraform state commands allow you to inspect and manage the associations between Terraform resource addresses and infrastructure objects.

They are useful for controlled state operations that cannot be achieved simply by changing resource arguments.

## 2. Purpose — Why do we use State Commands?

State commands help with:

- Inspecting managed resources.
- Moving resources between addresses.
- Removing a resource association from state.
- Retrieving state snapshots.
- Supporting infrastructure refactoring and recovery.

## 3. Working — How do they work?

Common commands include:

| Command | Purpose |
|---|---|
| terraform state list | List resource addresses |
| terraform state show ADDRESS | Show recorded resource details |
| terraform state mv SOURCE DESTINATION | Move a resource address in state |
| terraform state rm ADDRESS | Remove an association from state |
| terraform state pull | Retrieve the current state snapshot |
| terraform state push FILE | Push a state snapshot, subject to safety checks |

These commands can affect how Terraform manages infrastructure. Use them carefully and back up state before

potentially disruptive operations.

## 4. Practical Example — Rename a Resource Address

Suppose you rename the resource in your configuration from:

```
resource "aws_instance" "web" {
  # Existing configuration
}
```

to:

```
resource "aws_instance" "application" {
  # Same intended infrastructure configuration
}
```

Without migrating the address, Terraform may interpret the old resource as removed and the new resource as requiring creation.

Move the existing state address:

```
terraform state mv aws_instance.web aws_instance.application
```

Then run:

```
terraform plan
```

Review the result to confirm that Terraform recognizes the existing resource under its new address.

For modern configuration-driven refactoring, Terraform also supports moved blocks, which can document resource-address changes in code.

Important: terraform state rm

```
terraform state rm aws_instance.web
```

This removes the resource association from state but does not itself destroy the actual EC2 instance.

If the configuration still declares the resource, a subsequent plan may propose creating another instance because Terraform no longer associates the existing instance with that address.

## 5. Scenario-Based Interview Questions

### Q1. You rename a resource block and Terraform plans to destroy and recreate the instance. How would you avoid unnecessary replacement?

**Answer:** I would use terraform state mv or a suitable moved block to preserve the existing resource's association with its new address, then review the plan.

### Q2. Does terraform state rm delete the resource from AWS?

**Answer:** No. It removes the association from Terraform state. The infrastructure object remains in AWS unless separately deleted.

### Q3. When would you use terraform state push?

**Answer:** Only in a carefully controlled recovery or state-management procedure. I would verify the state snapshot, back up the current state, and follow the backend's safety checks before pushing.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** State commands ಬಳಸಿಕೊಂಡು Terraform state ಒಳಗಿನ resource records ನೋಡಬಹುದು ಅಥವಾ ಸರಿಪಡಿಸಬಹುದು. state rm ಮಾಡಿದರೆ AWS resource delete ಆಗುವುದಿಲ್ಲ; Terraformನ stateನಿಂದ ಅದರ association ಮಾತ್ರ ತೆಗೆದುಹಾಕುತ್ತದೆ. ತಪ್ಪಾಗಿ ಮಾಡಿದರೆ ಮುಂದೆ duplicate resources create ಆಗುವ ಅಪಾಯವಿದೆ.

# Topic 11: Terraform Workspaces and State Separation

## 1. Definition — What are Terraform Workspaces?

Terraform CLI workspaces allow one configuration and working directory to use separate state instances.

The default workspace is named default. Additional workspaces can be created for other state instances.

## 2. Purpose — Why do we use Workspaces?

Workspaces can be useful when the same configuration must manage separate instances of similar infrastructure.

For example, you might use different workspaces for isolated test environments.

However, workspaces alone do not provide separate cloud accounts, credentials, or access-control boundaries.

## 3. Working — How do Workspaces work?

Common commands:

```
terraform workspace list
terraform workspace new dev
terraform workspace select dev
terraform workspace show
```

Terraform selects the workspace and uses its corresponding state instance.

When using the S3 backend, state paths for workspaces are handled according to the backend's workspace conventions.

## 4. Practical Example

A configuration may use the selected workspace to determine an environment label:

```
locals {
  environment = terraform.workspace
}

resource "aws_s3_bucket" "app" {
  bucket = "company-app-${local.environment}-12345"

  tags = {
    Environment = local.environment
  }
}
```

This is an illustrative naming pattern; the bucket name must be globally unique and valid.

Important: Do not rely on workspaces alone to protect production infrastructure. For environments requiring strong isolation, separate configurations, state keys, AWS accounts, and access policies may be more appropriate.

## 5. Scenario-Based Interview Questions

### Q1. What is the difference between a Terraform workspace and a Git branch?

**Answer:** A Git branch tracks a version of source code. A Terraform workspace provides a separate state instance for a configuration. They solve different problems.

### Q2. Can workspaces automatically isolate development and production AWS accounts?

**Answer:** No. Workspaces separate state instances, but account selection and security boundaries must be configured separately.

### Q3. When would you use separate root configurations instead of workspaces?

**Answer:** When environments differ significantly or require stronger separation in credentials, permissions, infrastructure architecture, or deployment processes.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** Workspace ಒಂದೇ Terraform configurationಗೆ ಬೇರೆ stateಗಳನ್ನು ಬಳಸಲು ಸಹಾಯ ಮಾಡುತ್ತದೆ. ಆದರೆ dev ಮತ್ತು prod ಬೇರೆ workspaceಗಳಲ್ಲಿ ಇವೆ ಅಂದ ಮಾತ್ರಕ್ಕೆ ಅವು ಬೇರೆ AWS accounts ಆಗುವುದಿಲ್ಲ. Account ಮತ್ತು permissions ಅನ್ನು ಪ್ರತ್ಯೇಕವಾಗಿ configure ಮಾಡಬೇಕು.

## Topic 12: Real-Time Production Scenarios

### Scenario 1: Jenkins reports that Terraform state is locked

1. Definition: A Terraform operation cannot acquire the required state lock.
2. Purpose: Locking protects shared state from conflicting Terraform operations.
3. Working: Terraform checks whether the configured backend permits the operation to acquire the lock.
4. Practical Example:

```
Error: Error acquiring the state lock
```

Troubleshooting:

1. Check whether another Jenkins build is running.
2. Check whether a developer is running Terraform against the same state.
3. Wait for a legitimate operation to finish.
4. Investigate whether the lock is stale.
5. Use terraform force-unlock only when you have verified that the lock is no longer active.

5. Scenario-Based Interview Question

### How would you fix a stale state lock in production?

**Answer:** I would identify the operation that owns the lock and confirm that it has stopped. I would then follow the backend's recovery procedure and force-unlock only when necessary. I would run a fresh plan before applying changes.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** ಮೊದಲು Jenkinsನಲ್ಲಿ ಬೇರೆ build ನಡೆಯುತ್ತಿದೆಯಾ ಅಂತ ನೋಡಬೇಕು. Active operation ಇದ್ದರೆ ಕಾಯಬೇಕು. Lock stale ಅಂತ ಖಚಿತವಾದ ನಂತರ ಮಾತ್ರ unlock ಮಾಡಬೇಕು.

### Scenario 2: Terraform state file is missing

1. Definition: Terraform cannot find or access the state information required for the current configuration.
2. Purpose: Recover the correct resource associations before performing further infrastructure operations.

3. Working: Check whether the state exists in a remote backend, backup, or previous S3 object version.

4. Practical Example:

```
terraform state list
terraform state pull
terraform plan
```

Use these commands only after confirming the configured backend and understanding the current state condition. If state is missing, state list and state pull may fail or return information that does not represent the lost state.

5. Scenario-Based Interview Question

**Your local state file has disappeared, but the AWS resources still exist. What do you do?**

> **Answer:** I would stop applies, check remote state and backups, and restore the correct state. If recovery is impossible, I would carefully import existing resources or reconstruct state through a controlled procedure. I would not blindly apply the configuration.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** State file ಇಲ್ಲ ಅಂದರೆ AWS resources ಇಲ್ಲ ಅಂತಲ್ಲ. ಮೊದಲು backup ಅಥವಾ remote state ಹುಡುಕಬೇಕು. ಸರಿಯಾದ state ಸಿಗದಿದ್ದರೆ existing resources ಅನ್ನು ಗುರುತಿಸಿ controlled recovery ಮಾಡಬೇಕು.

## Scenario 3: Jenkins and local Terraform show different plans

1. Definition: Two execution environments produce different proposed infrastructure changes.

2. Purpose: Identify differences in state, configuration, provider versions, variables, or AWS authentication.

3. Working: Compare the environment settings and ensure both executions use the intended configuration and state.

4. Practical Example:

```
terraform version
terraform providers
terraform workspace show
terraform plan
```

Also verify the backend key, AWS account, region, branch, variable files, and lock file.

5. Scenario-Based Interview Question

**Terraform works locally, but Jenkins proposes creating duplicate EC2 instances. What would you check?**

> **Answer:** I would check whether Jenkins uses the same remote backend and state key, workspace, branch, variables, and AWS account as my local environment. I would stop the apply until the discrepancy is resolved.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** Local ಮತ್ತು Jenkins ಒಂದೇ state ಬಳಸುತ್ತಿವೆಯಾ ಅನ್ನೋದನ್ನು ಮೊದಲು ನೋಡಬೇಕು. ಬೇರೆ state key ಅಥವಾ account ಇದ್ದರೆ ಈಗಾಗಲೇ ಇರುವ instance Terraformಗೆ ಕಾಣದೇ duplicate create ಆಗುವ ಸಾಧ್ಯತೆ ಇದೆ.

## Scenario 4: Someone accidentally overwrites the S3 state object

1. Definition: The stored state no longer represents the correct resource associations because it was overwritten or corrupted.

2. Purpose: Recover the appropriate state while preserving the current evidence and avoiding additional damage.

3. Working: Identify the affected object, preserve the current version, inspect S3 version history, and restore the correct state using an approved recovery process.

4. Practical Example:

The backend uses an object such as:

```
s3://company-terraform-state/prod/eks/terraform.tfstate
```

Check the bucket's version history and determine which state version is appropriate. Do not assume the most recent previous version is automatically correct.

5. Scenario-Based Interview Question

**How would you recover a state object accidentally overwritten in S3?**

> **Answer:** I would pause Terraform operations, preserve the current object version, inspect available versions and incident timing, and restore the version that correctly represents the infrastructure. I would verify the restored state and review a fresh plan before resuming normal operations.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** S3 versioning ಮುಂಚೆಯೇ enable ಆಗಿದ್ದರೆ ಹಿಂದಿನ versions ಸಿಗಬಹುದು. ಸರಿಯಾದ version ಆಯ್ಕೆ ಮಾಡಿ restore ಮಾಡಬೇಕು. Restore ಮಾಡಿದ ನಂತರ resources ಮತ್ತು state match ಆಗುತ್ತಿವೆಯಾ ಅಂತ ಪರಿಶೀಲಿಸಬೇಕು.

### Scenario 5: Terraform plans to destroy an important resource

1. Definition: Terraform proposes deleting a resource because of configuration changes, state changes, or other lifecycle requirements.

2. Purpose: Prevent accidental data loss and unexpected service interruption.

3. Working: Inspect the planned actions, identify why destruction is proposed, and verify resource ownership and dependencies.

4. Practical Example:

```
terraform plan
```

If the plan includes unexpected destruction, do not apply it until the reason is understood.

5. Scenario-Based Interview Question

**Terraform plans to destroy your production database. What is your response?**

> **Answer:** I would stop the deployment, inspect the resource change and its dependencies, check the configuration and state, and verify whether the deletion is intentional. I would require the appropriate review, backup, and approval before proceeding.

> **ಸರಳ ಕನ್ನಡ ವಿವರಣೆ:** Production database delete ಆಗುವ plan ಕಂಡರೆ ತಕ್ಷಣ apply ಮಾಡಬಾರದು. ಯಾಕೆ delete ಆಗುತ್ತಿದೆ ಅಂತ ಮೊದಲು ಕಂಡುಹಿಡಿದು, backup ಮತ್ತು approval ಪರಿಶೀಲಿಸಬೇಕು.

## Quick Revision — Terraform State Commands

| Command | Purpose |
|---|---|
| terraform state list | List resources in state |
| terraform state show ADDRESS | Inspect a resource in state |
| terraform state pull | Retrieve a state snapshot |
| terraform state mv SOURCE DESTINATION | Move a resource address |
| terraform state rm ADDRESS | Remove a resource association from state |
| terraform plan | Preview configuration-driven changes |
| terraform plan -refresh-only | Preview state updates based on refreshed infrastructure information |
| terraform apply -refresh-only | Apply refresh-only state updates |
| terraform import ADDRESS ID | Import an existing supported object |
| terraform workspace list | List Terraform workspaces |
| terraform init | Initialize or reinitialize the backend and providers |

## Final study checklist

Part 6 study tracker: 0/11 completed

- [ ] Terraform State and its purpose
- [ ] Local State vs Remote State
- [ ] Backends and S3 configuration
- [ ] State locking and stale-lock recovery
- [ ] Refresh and drift detection
- [ ] Importing existing resources
- [ ] State backups and recovery
- [ ] State commands and resource-address changes
- [ ] Terraform workspaces and environment isolation
- [ ] Jenkins and remote state troubleshooting
- [ ] Production recovery scenarios

### Key interview takeaway

Remember these five points:

1. State tracks Terraform's association with infrastructure.
2. Backend determines where state is stored.
3. Locking helps prevent conflicting Terraform operations.
4. Refresh and planning help identify differences between configuration and actual infrastructure.
5. Backup and recovery protect against state loss and accidental changes.

Best practice: For your AWS Terraform and Jenkins projects, use a properly secured S3 backend, enable S3 versioning, configure supported state locking, restrict IAM permissions, and always review the plan before production changes.